



```
async def deploy_pipeline():
    project = "AI"
    env = "stg"

    → DeployResult:
        agent = CodeAgent(model="gpt-4o")
        await agent.run(project)
        tests = await agent.run_tests()
        if tests.passed:
            return await agent.deploy(env)

const pipeline = new AIPipeline({
  model: "Claude-sonnet",
  tools: ["bash", "editor", "browser"],
  maxIterations: 50
})

await pipeline.execute()
# Construa um sistema completo
# de gestão de pedidos com
# testes unitários e deploy
```

— O PERITO DE CUSTOS COM IA —

PLAYBOOK — ECONOMIA DE TOKENS

◆ PARA INICIANTES

16 capítulos

projetos práticos



AUTOR

Heverton Eduardo Peres

Especialista em Marketing e Desenvolvimento de Soluções

16
CAPÍTULOS

70+
PÁGINAS

∞
PROJETOS

Objetivo do Material

Cada token consumido é dinheiro em movimento: o leitor aprende a enxergar a máquina de custos LLM como um sistema de fluxo de caixa que pode ser otimizado em 8 pontos táticos e 4 pilares estratégicos, transformando-se de consumidor passivo a gestor consciente de despesa com IA.

Obra-mãe: economia-extrema-de-tokens

Passos práticos: 8

Você é: Engenheiro de Custos com IA

Sumário

Objetivo do Material	4
Como usar este playbook	5
Mapa dos Estágios	6
Passos Práticos	7
Passo 1 — A Crise de Custos com LLMs: De \$100 para \$10k Sem Aviso	7
Passo 2 — Como Funciona a Economia de Tokens: Caveman e Headroom	9
Passo 3 — Skills Agênticas de Compressão: Caveman, Headroom, Lean-CTX	13
Passo 4 — Cache Inteligente e Memory: RTK-Memory + LiteLLM	16
Passo 5 — Empacotamento de Contexto: Repomix	19
Passo 6 — Refatoração Estrutural com AST: ast-grep	21
Passo 7 — Otimização de Prompts: DSPy	23
Passo 8 — Governança em Produção: 4 Pilares + Checklist	25
Checklist Mestre	29

Objetivo do Material

Apresentar a crise silenciosa: projetos com LLMs que começam com \$100/mês e viram \$10k/mês sem que ninguém tenha visto vindo. Estabelecer que a economia de tokens não é mágica, mas engenharia. Mostrar o mapa do livro: 8 ferramentas que reduzem custo em pontos específicos (compressão, cache, memória, empacotamento, refatoração, otimização de prompt) e 4 pilares que sustentam tudo (mensuração, observabilidade, fallback, governança). O leitor vai sair do livro sabendo exatamente onde seu dinheiro vai e como não sair do orçamento.

Como usar este playbook

Você é o **Engenheiro de Custos com IA**. Cada passo é um card independente com sete partes: objetivo, pré-requisito, entregas, execução, gate de verificação, critério de conclusão e armadilhas.

Este documento **não repete a teoria** do livro. Quando precisar do porquê, siga a referência cruzada do card para o capítulo correspondente.

Mapa dos Estágios

#	Estágio	Passos
1	Token	1, 2, 3, 4
2	Custo	5, 6, 7, 8

Passos Práticos

Passo 1 — A Crise de Custos com LLMs: De \$100 para \$10k Sem Aviso

Estágio: Token · **Origem:** Cap. 1 — A Crise de Custos com LLMs: De \$100 para \$10k Sem Aviso

① Objetivo do passo

O leitor entende a urgência e o escopo do problema: tokens não são uma abstração, mas dinheiro circulando; a falta de visibilidade é o verdadeiro inimigo.

② Pré-requisito

Nenhum — este é o ponto de partida

③ Entregas

- `calculadora_projecao_custos.py`

④ Execução

Execução

```
"""
calculadora_projecao_custos.py
Ferramenta de bolso do Engenheiro de Custos com IA.

Uso:
python calculadora_projecao_custos.py chamada
python calculadora_projecao_custos.py projetar
python calculadora_projecao_custos.py auditar caminho/para/log.csv
"""

import csv
import sys

# Preço de referencia por 1 milhao de tokens (valores ilustrativos, em dolares).
# Troque pelos precos reais do seu provedor antes de usar em producao.
PRECO_ENTRADA_POR_MILHAO = 3.00
PRECO_SAIDA_POR_MILHAO = 15.00

def custo_de_uma_chamada(tokens_entrada: int, tokens_saida: int) -> float:
    """Calcula o custo de uma unica chamada de API, em dolares.
```

```

Cada tipo de token tem um preco diferente por milhao [8]: tokens de
saida custam mais porque exigem geracao ativa, nao so leitura.
"""

custo_entrada = (tokens_entrada / 1_000_000) * PRECO_ENTRADA_POR_MILHAO
custo_saida = (tokens_saida / 1_000_000) * PRECO_SAIDA_POR_MILHAO
return round(custo_entrada + custo_saida, 6)

def projetar_tres_meses(
    usuarios_mes1: int,
    chamadas_por_usuario_dia: int,
    tokens_medios_por_chamada: int,
    taxa_crescimento_contexto: float,
) -> list:
    """Projeta o custo mensal para 3 meses.

    A cada mes, o numero de usuarios dobra (cenario tipico de adocao
    inicial) e o contexto medio por chamada cresce pela taxa informada,
    simulando o efeito de "juros compostos" do historico reenviado.
    """

    resultados = []
    usuarios = usuarios_mes1
    tokens_por_chamada = tokens_medios_por_chamada

    for mes in range(1, 4):
        chamadas_no_mes = usuarios * chamadas_por_us

```

Execução

```

$ python calculadora_projecao_custos.py projetar
Mes 1: 50 usuarios, 2000 tokens/chamada, custo $135.0
Mes 2: 100 usuarios, 3200 tokens/chamada, custo $432.0
Mes 3: 200 usuarios, 5120 tokens/chamada, custo $1382.4

```

Execução

```

def alertar_limite_orcamento(resumo_por_dia: dict, limite_diario_dolares: float) -> list:
    """Compara o custo de cada dia contra um limite de orcamento.

    Retorna a lista de dias que estouraram o limite, ordenada do maior
    excesso para o menor. Esta e a diferenca entre auditar depois do
    fato e ser avisado no mesmo dia em que o custo sai da faixa esperada.
    """

    dias_fora_do_orcamento = []
    for data, custo_do_dia in resumo_por_dia.items():
        if custo_do_dia > limite_diario_dolares:
            dias_fora_do_orcamento.append((data, custo_do_dia))
    return sorted(dias_fora_do_orcamento, key=lambda item: item[1], reverse=True)

```

Execução

```

def ponto_de_equilibrio_saas_vs_api(
    custoassinatura_mensal_por_usuario: float,
    numero_de_usuarios: int,
    custo_medio_por_chamada: float,
) -> int:
    """Calcula quantas chamadas por mes, no total, igualam o custo

```

de uma assinatura SaaS de preço fixo por usuário.

Acima desse número de chamadas, pagar por token (modelo de API) fica mais caro que a assinatura; abaixo, fica mais barato.

"""

```
if custo_medio_por_chamada <= 0:
    raise ValueError("custo_medio_por_chamada deve ser maior que zero")
custo_total_assinatura = custo_assinatura_mensal_por_usuario * numero_de_usuarios
return int(custo_total_assinatura / custo_medio_por_chamada)
```

⑤ Verificação / Gate

```
python calculadora_projecao_custos.py auditar caminho/para/log.csv
```

⑥ Feito quando...

- ☐ Exporte (ou simule) um CSV com as colunas `data,tokens_entrada,tokens_saida,modelo` das últimas duas semanas do seu projeto
- ☐ Rode `calculadora_projecao_custos.py auditar` sobre esse CSV e identifique o dia de maior custo
- ☐ Calcule quantos tokens de entrada, em média, são histórico reenviado versus pergunta nova
- ☐ Rode o modo `projetar` com os parâmetros reais do seu projeto e compare com a fatura real dos últimos 2 meses
- ☐ Escreva, em uma frase, o estágio da curva de escalada em que seu projeto está hoje (Mês 1, 2 ou 3 da seção Ilustra)
- ☐ Rode `ponto_de_equilibrio_saas_vs_api` com o preço de uma assinatura SaaS equivalente ao seu caso de uso e descubra se pagar por token ainda é a opção mais barata no seu volume atual

⑦ Armadilhas

- (a completar)

Passo 2 — Como Funciona a Economia de Tokens: Caveman e Headroom

Estágio: Token · **Origem:** Cap. 2 — Como Funciona a Economia de Tokens: Caveman e Headroom

① Objetivo do passo

O leitor entende que a economia de tokens não é ajuste fino, mas arquitetura: onde cortar, quando sumarizar, e como medir o impacto de cada corte.

② Pré-requisito

Passo 1 concluído

③ Entregas

- output/**

④ Execução

Medindo o índice de compressão do seu próprio texto

```
# medir_compressao.py
# Estima tokens de um texto (aproximacao: 4 caracteres = 1 token)
# e calcula o indice de compressao entre uma versao original e uma versao comprimida.

def estimar_tokens(texto: str) -> int:
    # Aproximacao padrao de mercado para estimativa rapida sem chamar API
    return max(1, len(texto) // 4)

def indice_compressao(original: str, comprimido: str) -> float:
    tokens_originais = estimar_tokens(original)
    tokens_comprimidos = estimar_tokens(comprimido)
    reducao = 1 - (tokens_comprimidos / tokens_originais)
    return round(reducao * 100, 1)

if __name__ == "__main__":
    original = (
        "Ola! Gostaria muito de pedir, se possivel, que voce me ajudasse "
        "a entender por que o teste esta falhando, pois isso esta me "
        "deixando bastante preocupado com o prazo do projeto."
    )
    comprimido = "Teste falhando. Causa: prazo em risco."

    print(f"Tokens originais: {estimar_tokens(original)}")
    print(f"Tokens comprimidos: {estimar_tokens(comprimido)}")
    print(f"Indice de compressao: {indice_compressao(original, comprimido)}%")
```

Caveman: aplicando as 3 regras de corte

```
# caveman_corte.py
# Aplica 3 regras deterministicas de corte a um paragrafo de raciocinio verboso.

import re

SAUDACOES = ["ola", "bom dia", "boa tarde", "gostaria de", "vou analisar"]

def remover_saudacao(texto: str) -> str:
    linhas = texto.split(". ")
    filtradas = [l for l in linhas if not any(s in l.lower() for s in SAUDACOES)]
    return ". ".join(filtradas)

def remover_repeticao_do_prompt(texto: str, prompt_usuario: str) -> str:
    if prompt_usuario and prompt_usuario.lower() in texto.lower():
        texto = texto.lower().replace(prompt_usuario.lower(), "")
    return texto.strip(" .")
```

```
def encurtar_frases_longas(texto: str, limite_palavras: int = 20) -> str:
    frases = re.split(r"(?<=[.!?]) ", texto)
    curtas = []
    for frase in frases:
        palavras = frase.split()
        if len(palavras) > limite_palavras:
            metade = len(palavras) // 2
            curtas.append(" ".join(palavras[:metade]) + ".")
            curtas.append(" ".join(palavras[metade:]))
        else:
            curtas.append(frase)
    return " ".join(curtas)

def aplicar_caveman(texto: str, prompt_usuario: str = "") -> str:
    etapa1 = remover_saudacao(texto)
    etapa2 = remover_repeticao_do_prompt(etapa1, prompt_usuario)
    etapa3 = encurtar_frases_longas(etapa2)
    return etapa3

if __name__ == "__main__":
    bruto = (
        "Ola! Vou analisar cuidadosamente sua solicitacao de corrigir "
        "o teste que falhou porque o mock retornou nulo em vez do "
        "objeto esperado, e isso quebrou a assertão da linha 42."
    )
    print(aplicar_caveman(bruto))
```

Headroom: a regra 3+4 em produção

```
#!/usr/bin/env bash
# headroom.sh – comprime saída de um comando quando ela passa de 7 linhas
# Uso: ./headroom.sh "npm test"

set -euo pipefail

SAIDA=$(eval "$1" 2>&1) || true
TOTAL_LINHAS=$(echo "$SAIDA" | wc -l)

if [ "$TOTAL_LINHAS" -gt 7 ]; then
    echo "$SAIDA" | head -n 3
    echo "... [$(($TOTAL_LINHAS - 7)) linhas omitidas pelo headroom] ..."
    echo "$SAIDA" | tail -n 4
else
    echo "$SAIDA"
fi
```

Projetando o efeito cascata do Pilar 4

```
# projetar_economia_mensal.py
# Projeta a economia mensal de tokens ao aplicar caveman + headroom
# em N execucoes diarias de build/raciocinio.

def estimar_tokens(texto_ou_linhas, chars_por_linha: int = 60) -> int:
    # Aproximacao: 4 caracteres = 1 token (ver ressalva na secao Explica)
    total_chars = texto_ou_linhas * chars_por_linha
    return max(1, total_chars // 4)
```

```
def projetar_economia_mensal(
    linhas_log_bruto: int,
    execucoes_por_dia: int,
    tokens_raciocinio_bruto: int,
    tokens_raciocinio_caveman: int,
    dias_uteis_mes: int = 22,
) -> dict:
    linhas_log_comprimido = 7 # regra 3 + 4 do headroom
    tokens_log_bruto = estimar_tokens(linhas_log_bruto)
    tokens_log_comprimido = estimar_tokens(linhas_log_comprimido)

    economia_log_por_execucao = tokens_log_bruto - tokens_log_comprimido
    economia_raciocinio_por_execucao = tokens_raciocinio_bruto -
tokens_raciocinio_caveman

    economia_diaria = (
        economia_log_por_execucao + economia_raciocinio_por_execucao
    ) * execucoes_por_dia

    return {
        "tokens_economizados_por_execucao": economia_log_por_execucao
+ economia_raciocinio_por_execucao,
        "tokens_economizados_por_dia": economia_diaria,
        "tokens_economizados_por_mes": economia_diaria * dias_uteis_mes,
    }

if __name__ == "__main__":
    resultado = projetar_economia_mensal(
        linhas_log_bruto=340,
        execucoes_por_dia=50,
        tokens_raciocinio_bruto=180,
        tokens_raciocinio_caveman=22,
    )
    for chave, valor in resultado.items():
        print(f"{chave}: {valor:,}".replace(
```

⑤ Verificação / Gate

```
./headroom.sh "npm run build"
```

⑥ Feito quando...

- ☐ Rode `medir_compressao.py` com um parágrafo real que você escreveu para o agente esta semana
- ☐ Aplique `aplicar_caveman()` no mesmo parágrafo e compare o índice de compressão
- ☐ Rode `headroom.sh` sobre a saída de um build ou teste do seu projeto atual
- ☐ Rode `projetar_economia_mensal.py` com os números reais de execuções por dia da sua equipe
- ☐ Preencha a tabela de decisão do capítulo com pelo menos 2 situações reais do seu fluxo de trabalho
- ☐ Identifique 1 lugar onde você aplicaria headroom mas NÃO deveria (ex.: evidência de auditoria)
- ☐ Reescreva em modo caveman o próximo comentário de revisão de código que você receber, antes de repassá-lo ao agente

⑦ Armadilhas

- (a completar)

Passo 3 — Skills Agênticas de Compressão: Caveman, Headroom, Lean-CTX

Estágio: Token · **Origem:** Cap. 3 — Skills Agênticas de Compressão: Caveman, Headroom, Lean-CTX

① Objetivo do passo

O leitor monta seu próprio pipeline de compressão, escolhendo qual ferramenta encadear de acordo com o tipo de saída (log, código, referência) e o contexto disponível.

② Pré-requisito

Passo 2 concluído

③ Entregas

- `.claude/settings.json`

④ Execução

Pilar 1 — Simulando a economia da cadeia

```
# Simulador didatico de economia de tokens no pipeline caveman -> headroom -> lean-ctx
# Aproximacao: 4 caracteres ~= 1 token (suficiente para ilustrar a ordem de grandeza)

def estimar_tokens(texto):
    """Estima tokens dividindo o total de caracteres por 4."""
    return max(1, len(texto) // 4)

def economia_percentual(antes, depois):
    """Calcula quanto foi economizado, em percentual, de antes para depois."""
    if antes == 0:
        return 0.0
    return round((1 - (depois / antes)) * 100, 1)

# Estagio 1: raciocinio interno (caveman)
pensamento_verboso = (
    "Vou analisar cuidadosamente o problema apresentado pelo usuario, "
    "considerando todos os angulos possiveis antes de chegar a uma conclusao final."
)
pensamento_comprimido = "Analiso problema. Considero angulos. Concluo."

# Estagio 2: log de build (headroom) - simulando 40 linhas cortadas para 7
```

```
log_bruto = "linha de warning repetida\n" * 40 + "ERR0: variavel nao definida na linha 12"
log_comprimido = "\n".join(log_bruto.splitlines()[:3] + log_bruto.splitlines()[-4:])

# Estagio 3: leitura de arquivo (lean-ctx) - simulando 1500 linhas vs 25 linhas
arquivo_inteiro = "linha de codigo\n" * 1500
fatia_lida = "linha de codigo\n" * 25

estagios = [
    ("caveman (pensamento)", pensamento_verboso, pensamento_comprimido),
    ("headroom (log)", log_bruto, log_comprimido),
    ("lean-ctx (arquivo)", arquivo_inteiro, fatia_lida),
]

for nome, antes, depois in estagios:
    t_antes = estimar_tokens(antes)
    t_depois = estimar_tokens(depois)
    print(f"{nome}: {t_antes} -> {t_depois} tokens ({economia_percent
```

Pilar 2 — Grep antes de read na prática

```
$ wc -l servico_pagamentos.py
1500 servico_pagamentos.py

$ grep -n "def calcular_taxa_conversao" servico_pagamentos.py
118:def calcular_taxa_conversao(valor, moeda_origem, moeda_destino):

$ sed -n '118,143p' servico_pagamentos.py
def calcular_taxa_conversao(valor, moeda_origem, moeda_destino):
    # ... 25 linhas da funcao, e so isso ...
    return valor_convertido

$ echo "Linhas lidas: 25 de 1500 (98,3% do arquivo nunca entrou no contexto)"
Linhas lidas: 25 de 1500 (98,3% do arquivo nunca entrou no contexto)
```

Pilar 3 — Declarando a exceção de governança no hook

```
{
  "hooks": {
    "compressao_automatica": {
      "ativar_para": ["logs/**", "src/**"],
      "skills": ["headroom", "lean-ctx"],
      "excecoes": {
        "nunca_comprimir": ["output/**", "auditoria/**", "*.pii.log"],
        "motivo": "Dados de obra e trilhas de auditoria exigem leitura integral, nunca resumida."
      }
    }
  }
}
```

Pilar 4 — Auditando a disciplina do grep-antes-de-read

```
# Auditor simples de disciplina lean-ctx: sinaliza leitura de arquivo grande sem grep previo
LIMITE_LINHAS_SEM_GREP = 50
```

```

eventos_sessao = [
    {"acao": "grep", "arquivo": "servico_pagamentos.py"},
    {"acao": "read", "arquivo": "servico_pagamentos.py", "linhas": 25},
    {"acao": "read", "arquivo": "config_geral.py", "linhas": 320}, # sem grep antes ->
violacao
    {"acao": "grep", "arquivo": "modelo_usuario.py"},
    {"acao": "read", "arquivo": "modelo_usuario.py", "linhas": 40},
]

def auditar_disciplina(eventos):
    """Retorna a lista de leituras que violaram a regra grep-antes-de-read."""
    arquivos_pesquisados = set()
    violacoes = []
    for evento in eventos:
        if evento["acao"] == "grep":
            arquivos_pesquisados.add(evento["arquivo"])
        elif evento["acao"] == "read":
            teve_grep_previo = evento["arquivo"] in arquivos_pesquisados
            arquivo_grande = evento.get("linhas", 0) > LIMITE_LINHAS_SEM_GREP
            if arquivo_grande and not teve_grep_previo:
                violacoes.append(evento["arquivo"])
    return violacoes

violacoes = auditar_disciplina(eventos_sessao)
print(f"Leituras sem grep previo em arquivo grande: {violacoes}")
# Saída esperada: ['config_geral.py']

```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Escolha um log de build recente do seu projeto com mais de 30 linhas e aplique manualmente a regra 3+4 do headroom; confira se a causa raiz continua visível
- ☐ Rode `grep -n` para localizar uma função específica em um arquivo com mais de 200 linhas do seu projeto e leia só a fatia correspondente
- ☐ Escreva no seu `.claude/settings.json` (ou equivalente) uma exceção de path que nunca deve passar por compressão automática
- ☐ Calcule, usando o script do Pilar 1, a economia percentual real de um dos seus próprios logs
- ☐ Rode `grep -n` para um símbolo do seu projeto que você suspeita ter mais de uma ocorrência; se houver ambiguidade, refine a busca por diretório ou contexto antes de decidir qual trecho ler
- ☐ Aplique o auditor do Pilar 4 (ou uma versão simplificada dele) a um log real da sua última sessão de agente e identifique se alguma leitura de arquivo grande pulou o grep

⑦ Armadilhas

- (a completar)

Passo 4 — Cache Inteligente e Memory: RTK-Memory + LiteLLM

Estágio: Token · **Origem:** Cap. 4 — Cache Inteligente e Memory: RTK-Memory + LiteLLM

① Objetivo do passo

O leitor configura cache real (não teórico) no seu fluxo de trabalho com IA, vê o desconto materializar e entende quando reutilizar contexto compensa mais que gerá-lo do zero.

② Pré-requisito

Passo 3 concluído

③ Entregas

- CLAUDE.md
- AGENTS.md
- RTK-SCRATCHPAD.md

④ Execução

Medindo cache hit com um script simples

```
# medir_cache_hit.py
# Script simples, comentado passo a passo, para estimar a taxa de cache hit
# a partir de um log de prompts (cada item da lista simula uma chamada).

def prefixo_estavel(prompt: str, tamanho: int = 40) -> str:
    """Retorna os primeiros N caracteres do prompt.

    Na vida real, o 'prefixo estavel' e o system prompt (CLAUDE.md/AGENTS.md).
    Aqui simplificamos pegando o inicio do texto para fins didaticos.
    """
    return prompt[:tamanho]

def medir_taxa_cache_hit(log_de_chamadas: list) -> dict:
    """Conta quantas chamadas repetem o mesmo prefixo da chamada anterior.

    Retorna um dicionario com o total de chamadas, os hits (prefixo repetido)
    e a taxa de acerto em percentual.
    """
    total = len(log_de_chamadas)
    hits = 0
    prefixo_anterior = None

    for prompt in log_de_chamadas:
        prefixo_atual = prefixo_estavel(prompt)
        if prefixo_atual == prefixo_anterior:
            hits += 1
        prefixo_anterior = prefixo_atual
```

```

    taxa = (hits / total * 100) if total > 0 else 0.0
    return {"total_chamadas": total, "cache_hits": hits, "taxa_hit_percentual":
round(taxa, 1)}

if __name__ == "__main__":
    # Cenário A: prompt-base fixo (RTK-Memory ativo) + pergunta variavel no final
    log_com_rtk_memory = [
        "SYSTEM: Voce e um assistente de codigo. PERGUNTA: como criar uma lista?",
        "SYSTEM: Voce e um assistente de codigo. PERGUNTA: como ler um arquivo?",
        "SYSTEM: Voce e um assistente de codigo. PERGUNTA: como somar dois numeros?",
    ]

    resultado = medir_taxa_cache_hit(log_com_rtk_memory)

```

Configurando o LiteLLM como router com cache semântico

```

# litellm-config.yaml
model_list:
  - model_name: modelo-padrao
    litellm_params:
      model: anthropic/claude-3-5-sonnet
      api_key: "os.environ/ANTHROPIC_API_KEY"
  - model_name: modelo-alternativo
    litellm_params:
      model: openai/gpt-4o-mini
      api_key: "os.environ/OPENAI_API_KEY"

litellm_settings:
  cache: true
  cache_params:
    type: redis
    host: "localhost"
    port: 6379
    similarity_threshold: 0.95 # so reaproveita resposta acima de 95% de similaridade

```

Traduzindo taxa de cache hit em economia real

```

# estimar_economia_cache.py
# Projeta a economia mensal a partir da taxa de cache hit medida
# e do volume de chamadas do seu próprio fluxo.

def estimar_economia_mensal(
    taxa_hit_percentual: float,
    chamadas_por_dia: int,
    tokens_prefixo_medio: int,
    custo_por_1k_tokens: float,
    desconto_cache: float = 0.90,
) -> dict:
    """Estima quanto do custo de prefixo é evitado por mês graças ao cache.

    taxa_hit_percentual: saída de medir_taxa_cache_hit() (0 a 100)
    chamadas_por_dia: volume medio de chamadas do seu fluxo
    tokens_prefixo_medio: tamanho medio, em tokens, do prompt-base
    custo_por_1k_tokens: preco cheio por 1000 tokens processados
    desconto_cache: fracao do custo evitada em cada hit (0.90 = 90%)
    """

```

```

"""
chamadas_por_mes = chamadas_por_dia * 30
chamadas_com_hit = chamadas_por_mes * (taxa_hit_percentual / 100)

custo_cheio_por_chamada = (tokens_prefixo_medio / 1000) * custo_por_1k_tokens
economia_por_hit = custo_cheio_por_chamada * desconto_cache
economia_mensal_estimada = chamadas_com_hit * economia_por_hit

return {
    "chamadas_por_mes": chamadas_por_mes,
    "chamadas_com_cache_hit": round(chamadas_com_hit),
    "economia_mensal_estimada": round(economia_mensal_estimada, 2),
}

if __name__ == "__main__":
    # Exemplo: fluxo com 80% de cache hit (RTK-Memory ativo),
    # 500 chamadas/dia, prefixo de 2000 tokens, US$ 0,003 por 1k tokens
    resultado = estimar_economia_mensal(
        taxa_hit_percentual=80,
        chamadas_por_dia=500,
        tokens_prefixo_medio=2000,
        custo_por_1k_t

```

⑤ Verificação / Gate

```

docker run -d -p 4000:4000 -v $(pwd)/litellm-config.yaml:/app/config.yaml ghcr.io/berriai/litellm:main-latest --config /app/config.yaml

```

⑥ Feito quando...

- ❑ Rode o `medir_cache_hit.py` com um log real de prompts do seu projeto (substitua o exemplo pelas suas últimas 10-20 chamadas)
- ❑ Identifique, no seu `CLAUDE.md` / `AGENTS.md` atual, qualquer trecho que parece “aprendizado de sessão” e mova para um `RTK-SCRATCHPAD.md`
- ❑ Suba o `litellm-config.yaml` localmente e confirme que o cache semântico responde em poucos milissegundos numa segunda chamada parecida
- ❑ Documente, no seu repositório, a regra “o que muda no prompt-base e o que vai para o scratchpad”
- ❑ Rode o `estimar_economia_cache.py` com os números reais do seu fluxo (volume diário de chamadas, tamanho médio do prefixo, custo por 1k tokens do seu provedor) e registre o valor projetado como meta de acompanhamento mensal
- ❑ Se você opera mais de um agente com prompt próprio, liste onde há trechos de governança duplicados entre eles e planeje extraí-los para um módulo único e referenciado

⑦ Armadilhas

- (a completar)

Passo 5 — Empacotamento de Contexto: Repomix

Estágio: Custo · **Origem:** Cap. 5 — Empacotamento de Contexto: Repomix

① Objetivo do passo

O leitor reduz tamanho de contexto em 40-70% mantendo cobertura, criando snapshots reutilizáveis que viram entradas padrão para o agente.

② Pré-requisito

Passo 4 concluído

③ Entregas

- `.repomix.json`

④ Execução

Instalando e rodando o Repomix

```
$ npx repomix --style xml --output-show-line-numbers
$ cat repomix-output.xml | wc -c
```

Configurando regras próprias com `.repomix.json`

```
{
  "output": {
    "filePath": "repomix-output.xml",
    "style": "xml",
    "showLineNumbers": true
  },
  "include": [
    "src/**/*.py",
    "src/**/*.ts",
    "docs/**/*.md"
  ],
  "ignore": {
    "useGitignore": true,
    "customPatterns": [
      "**/*.test.fixture.*",
      "**/dist/**",
      "**/*.generated.*",
      "**/CHANGELOG.md"
    ]
  }
}
```

Medindo o ganho real: bytes e tokens estimados

```
# medir_ganho_repomix.py
# Compara o tamanho do snapshot compactado com a soma dos arquivos originais.
import os

CARACTERES_POR_TOKEN = 4 # heuristica aproximada para estimativa rapida

def tamanho_total_diretorio(caminho: str, extensoes: tuple) -> int:
    """Soma o tamanho em bytes de todos os arquivos com as extensoes dadas."""
    total = 0
    for raiz, _dirs, arquivos in os.walk(caminho):
        for nome in arquivos:
            if nome.endswith(extensoes):
                caminho_completo = os.path.join(raiz, nome)
                total += os.path.getsize(caminho_completo)
    return total

def estimar_tokens(bytes_totais: int) -> int:
    """Estimativa grosseira de tokens a partir do tamanho em bytes."""
    return bytes_totais // CARACTERES_POR_TOKEN

def comparar_ganho(caminho_projeto: str, caminho_snapshot: str) -> dict:
    """Retorna bytes/tokens do projeto original vs. do snapshot compactado."""
    bytes_original = tamanho_total_diretorio(caminho_projeto, (".py", ".ts", ".md"))
    bytes_snapshot = os.path.getsize(caminho_snapshot)

    return {
        "bytes_original": bytes_original,
        "bytes_snapshot": bytes_snapshot,
        "tokens_estimados_original": estimar_tokens(bytes_original),
        "tokens_estimados_snapshot": estimar_tokens(bytes_snapshot),
        "reducao_percentual": round((1 - bytes_snapshot / bytes_original) * 100, 1)
        if bytes_original > 0
        else 0.0,
    }

if __name__ == "__main__":
    import argparse
    import sys

    parser = argparse.ArgumentParser(description="Mede o ganho de empacotamento do Repomix.")
```

Travando o orçamento de tokens em CI

```
# .github/workflows/orcamento-contexto.yml
name: Orcamento de Contexto (Repomix)
on: [pull_request]

jobs:
  medir-snapshot:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Gerar snapshot Repomix
```

```
run: npx repomix --style xml --output-show-line-numbers
- name: Validar orçamento de tokens
run: |
    python medir_ganho_repomix.py --limite-tokens 60000 --falhar-se-exceder
```

⑤ Verificação / Gate

```
npx repomix --style xml --output-show-line-numbers
```

⑥ Feito quando...

- ☐ Instale o Repomix com `npx repomix --style xml --output-show-line-numbers` no seu projeto atual
- ☐ Escreva um `.repomix.json` com pelo menos 3 regras de `customPatterns` específicas do seu projeto
- ☐ Rode `medir_ganho_repomix.py` e registre a redução percentual real obtida
- ☐ Versione o `.repomix.json` no repositório e documente, em uma linha, quando ele deve ser revisado
- ☐ Configure o gate de CI de orçamento de tokens (seção Técnica) com um limite calibrado pela calculadora de custo do Capítulo 1, não por um número redondo escolhido de improviso

⑦ Armadilhas

- (a completar)

Passo 6 — Refatoração Estrutural com AST: ast-grep

Estágio: Custo · **Origem:** Cap. 6 — Refatoração Estrutural com AST: ast-grep

① Objetivo do passo

O leitor escreve uma query AST uma vez e reduz seu código em 20-30%, eliminando duplicação que o grep tradicional não vê. Menos código = menos contexto = menos custo.

② Pré-requisito

Passo 5 concluído

③ Entregas

- (a completar)

④ Execução

Provando a diferença entre grep e AST na prática

```
# comparar_grep_vs_ast.py
# Demonstracao didatica: contar chamadas "api.get(url)" via regex (texto)
# versus via arvore de sintaxe (estrutura), no mesmo trecho de codigo Python.

import ast
import re

codigo_fonte = """
resultado = api.get(url)
outro = api.get(
    url,
)
"""

def contar_via_regex(codigo: str) -> int:
    """Conta ocorrencias comparando texto, byte a byte, contra um padrao fixo."""
    padrao = re.compile(r"api\\.get\\(url\\)")
    return len(padrao.findall(codigo))

def contar_via_ast(codigo: str) -> int:
    """Conta ocorrencias comparando a estrutura da arvore de sintaxe.

    Encontra qualquer chamada no formato api.get(...), nao importa
    como ela foi formatada no arquivo original.
    """
    arvore = ast.parse(codigo)
    total = 0
    for no in ast.walk(arvore):
        if isinstance(no, ast.Call) and isinstance(no.func, ast.Attribute):
            se_e_api = isinstance(no.func.value, ast.Name) and no.func.value.id == "api"
            if se_e_api and no.func.attr == "get":
                total += 1
    return total

if __name__ == "__main__":
    print("Via regex (texto):", contar_via_regex(codigo_fonte))
    print("Via AST (estrutura):", contar_via_ast(codigo_fonte))
```

Investigação pontual sem escrever regra alguma

```
$ sg run -p 'api.get($URL)' --lang js
src/pedidos.js:12: resultado = api.get(url)
src/relatorios.js:47: dados = api.get(endpointRelatorio)
2 correspondencias em 2 arquivos, 8 ms
```

Reescrevendo em lote: da regra YAML ao repositório inteiro

```
$ npm install -g @ast-grep/cli
$ sg --version
0.30.0
```

Colocando um auditor automático no CI

```
# regras/proibir-console-log.yml
id: proibir-console-log-em-producao
language: JavaScript
rule:
  pattern: "console.log($$$$ARGS)"
```

⑤ Verificação / Gate

```
npm install -g @ast-grep/cli
```

⑥ Feito quando...

- ☐ Instale o ast-grep no seu ambiente (`npm install -g @ast-grep/cli` ou o binário nativo da plataforma)
- ☐ Rode `sg run -p '<um padrão do seu projeto>'` e compare o resultado com o mesmo `grep` sobre o mesmo padrão
- ☐ Escreva uma regra YAML de reescrita para uma mudança de assinatura real do seu código e rode com `--update-all` num branch isolado
- ☐ Adicione um step de `sg scan --error` num workflow de CI, mesmo que só em modo de aviso no início

⑦ Armadilhas

- (a completar)

Passo 7 — Otimização de Prompts: DSPy

Estágio: Custo · **Origem:** Cap. 7 — Otimização de Prompts: DSPy

① Objetivo do passo

O leitor vira de quem copia prompts prontos para quem otimiza seus próprios prompts, economizando 20-40% em tokens de prompt + reduzindo hallucinations.

② Pré-requisito

Passo 6 concluído

③ Entregas

- (a completar)

④ Execução

Execução

```

import dspy

# 1. Signature: contrato de entrada/saida (o que preencher, nao como pensar)
class ExtrairDadosContrato(dspy.Signature):
    """Extrai nome do cliente, valor e data de vencimento de um texto de contrato."""
    texto_contrato: str = dspy.InputField()
    nome_cliente: str = dspy.OutputField()
    valor: str = dspy.OutputField()
    vencimento: str = dspy.OutputField()

# 2. Module: a estrategia de chamada que cumpre esse contrato
class ExtratorDeContrato(dspy.Module):
    def __init__(self):
        super().__init__()
        self.extrair = dspy.Predict(ExtrairDadosContrato)

    def forward(self, texto_contrato):
        return self.extrair(texto_contrato=texto_contrato)

# 3. Exemplos de treino: poucos casos ja revisados manualmente bastam
exemplos_treino = [
    dspy.Example(
        texto_contrato="Contrato entre ACME LTDA e Joao Silva, valor R$ 4.500,00,
vencimento 10/09/2026",
        nome_cliente="Joao Silva",
        valor="R$ 4.500,00",
        vencimento="10/09/2026",
    ).with_inputs("texto_contrato"),
    # ... mais 4 a 9 exemplos revisados a mao formam um conjunto de treino razoavel
]

# 4. Metrica: funcao objetiva que diz se a saida esta certa
def validador_extracao(exemplo, predicao, trace=None):
    return (
        predicao.nome_cliente.strip() == exemplo.nome_cliente.strip()
        and predicao.valor.strip() == exemplo.valor.strip()
    )

# 5. Compilador: busca o prompt mais barato que ainda passa na metrica
from dspy.teleprompt import BootstrapFewShot

compilador = BootstrapFewShot(metric=validador_extracao)
extrator_otimizado = compil

```

Execução

```

# Passo extra: auditar o prompt que o compilador produziu antes de ir para producao
# (nao para editar a mao - para confirmar que faz sentido e nao vaza dado sensivel)
extrator_otimizado(
    texto_contrato="Contrato entre Beta ME e Carlos Nunes, valor R$ 900,00, vencimento
01/11/2026"
)

# Mostra a ultima chamada feita ao modelo, incluindo o prompt final montado

```

```
dspy.inspect_history(n=1)

# Boa pratica: salvar o modulo compilado em disco para nao recompilar a cada deploy
extrator_otimizado.save("extrator_contrato_compilado.json")

# Em producao, so carregar o artefato ja compilado - sem chamar o compilador de novo
extrator_producao = ExtratorDeContrato()
extrator_producao.load("extrator_contrato_compilado.json")
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Escolha um prompt do seu projeto que roda pelo menos algumas centenas de vezes por mês
- ☐ Escreva a Signature dele: quais campos realmente entram e quais devem sair
- ☐ Separe de 5 a 10 casos reais já revisados manualmente como exemplos de treino
- ☐ Escreva uma função de métrica objetiva (não “parece certo”, mas um critério comparável)
- ☐ Rode o compilador e compare o tamanho do prompt final com o prompt manual atual
- ☐ Antes de colocar em produção, inspecione o prompt compilado (`dspy.inspect_history`) e confirme que nenhum exemplo de treino carrega dado sensível

⑦ Armadilhas

- (a completar)

Passo 8 — Governança em Produção: 4 Pilares + Checklist

Estágio: Custo · **Origem:** Cap. 8 — Governança em Produção: 4 Pilares + Checklist

① Objetivo do passo

O leitor monta um sistema de governança que sustenta redução de custos: automatiza o que pode, avisa o que precisa aprovação, e garante que ninguém gaste sem visibilidade.

② Pré-requisito

Passo 7 concluído

③ Entregas

- `log_custos.jsonl`
- `resumo_diario.py`

④ Execução

Middleware de mensuração e observabilidade

```
# middleware_custo.py
# Envolve uma chamada de LLM e registra custo, usuario e modelo por linha JSON.
import json
import time

PRECO_POR_1K_TOKENS = {
    "modelo-padrao": {"entrada": 0.003, "saida": 0.015},
    "modelo-barato": {"entrada": 0.0008, "saida": 0.004},
}

def calcular_custo(modelo: str, tokens_entrada: int, tokens_saida: int) -> float:
    """Calcula o custo estimado da chamada em dolares, a partir da tabela de precos."""
    preco = PRECO_POR_1K_TOKENS.get(modelo, PRECO_POR_1K_TOKENS["modelo-padrao"])
    custo_entrada = (tokens_entrada / 1000) * preco["entrada"]
    custo_saida = (tokens_saida / 1000) * preco["saida"]
    return round(custo_entrada + custo_saida, 6)

def registrar_chamada(usuario: str, modelo: str, tokens_entrada: int, tokens_saida: int,
arquivo="log_custos.jsonl"):
    """Grava um registro estruturado de uma chamada, pronto para agregacao posterior."""
    registro = {
        "timestamp": time.time(),
        "usuario": usuario,
        "modelo": modelo,
        "tokens_entrada": tokens_entrada,
        "tokens_saida": tokens_saida,
        "custo_usd": calcular_custo(modelo, tokens_entrada, tokens_saida),
    }
    with open(arquivo, "a", encoding="utf-8") as f:
        f.write(json.dumps(registro, ensure_ascii=False) + "\n")
    return registro

if __name__ == "__main__":
    exemplo = registrar_chamada("usuario_42", "modelo-padrao", tokens_entrada=1200,
tokens_saida=350)
    print("Chamada registrada:", exemplo)
```

Circuit breaker de custo

```
# circuit_breaker_custo.py
# Circuit breaker simples que desvia trafego quando o custo por chamada estoura o limite.
import time

class CircuitBreakerCusto:
    def __init__(self, limite_usd: float, janela_teste_segundos: int = 60):
        self.limite_usd = limite_usd
        self.janela_teste_segundos = janela_teste_segundos
        self.estado = "fechado"
        self.momento_abertura = None

    def registrar_custo(self, custo_usd: float) -> str:
        """Recebe o custo da ultima chamada e decide o proximo estado/rota."""
```

```

    if self.estado == "fechado":
        if custo_usd > self.limite_usd:
            self.estado = "aberto"
            self.momento_abertura = time.time()
            return "modelo-fallback"
        return "modelo-principal"

    if self.estado == "aberto":
        if time.time() - self.momento_abertura >= self.janela_teste_segundos:
            self.estado = "meio-aberto"
            return "modelo-principal" # testa uma amostra
        return "modelo-fallback"

    if self.estado == "meio-aberto":
        if custo_usd > self.limite_usd:
            self.estado = "aberto"
            self.momento_abertura = time.time()
            return "modelo-fallback"
        self.estado = "fechado"
        return "modelo-principal"

if __name__ == "__main__":
    breaker = CircuitBreakerCusto(limite_usd=0.05)
    print(breaker.registrar_custo(0.02)) # modelo-principal
    print(breaker.registrar_custo(0.09)) # modelo-fallback (abre)
    print(breaker.registrar_custo(0.03)) # aind

```

Fechando o buraco: piso de qualidade acoplado ao fallback

```

# circuit_breaker_qualidade.py
# Envolve o CircuitBreakerCusto com um piso de qualidade antes de aceitar o fallback.
from circuit_breaker_custo import CircuitBreakerCusto

def registrar_custo_com_qualidade(breaker: CircuitBreakerCusto, custo_usd: float,
resposta_valida: bool) -> str:
    """So aceita a rota de fallback se a amostra tambem passar num criterio minimo de
qualidade."""
    rota = breaker.registrar_custo(custo_usd)
    if rota == "modelo-fallback" and not resposta_valida:
        # custo baixo nao compensa se a resposta nao serve: forca nova tentativa no
principal
        breaker.estado = "aberto"
        return "modelo-principal-forcado"
    return rota

```

Checklist de produção e alerta automático

```

# checklist-governanca.yaml
# Checklist minimo de producao: uma linha por decisao que precisa de dono.
governanca_producao:
  aprovacao_de_modelo:
    exigida_acima_de_usd_mes: 500
    aprovador: "lider_tecnico_da_equipe"
  limite_de_gasto_por_projeto:
    valor_usd_mes: 2000

```

```
acao_ao_ultrapassar: "alerta + revisao obrigatoria"
auditoria_periodica:
  cadencia: "semanal"
  escopo: "reabrir 5% das chamadas e conferir custo real vs. estimado"
canais_de_alerta:
  - "slack:#custos-llm"
  - "email:financeiro@empresa.com"
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Implemente o `middleware_custo.py` sobre uma chamada real do seu projeto e rode por um dia inteiro
- ☐ Rode o `resumo_diario.py` sobre o log gerado e confira se o total agregado bate com o que você esperava gastar naquele dia
- ☐ Ajuste o `limite_usd` do `circuit_breaker_custo.py` para o teto que sua equipe consideraria aceitável por chamada
- ☐ Preencha o `checklist-governanca.yaml` com os nomes reais de aprovador, limite de gasto e canal de alerta da sua equipe
- ☐ Defina, por escrito, qual métrica de qualidade mínima seu circuit breaker vai checar antes de aceitar uma rota de fallback como segura
- ☐ Defina a cadência de recálculo do baseline de alerta (ex.: média móvel de 14 dias) para que o webhook não vire ruído ignorado

⑦ Armadilhas

- (a completar)

Checklist Mestre

Passo 1 — A Crise de Custos com LLMs: De \$100 para \$10k Sem Aviso

- ☐ Exporte (ou simule) um CSV com as colunas `data,tokens_entrada,tokens_saida,modelo` das últimas duas semanas do seu projeto
- ☐ Rode `calculadora_projecao_custos.py` auditar sobre esse CSV e identifique o dia de maior custo
- ☐ Calcule quantos tokens de entrada, em média, são histórico reenviado versus pergunta nova
- ☐ Rode o modo `projetar` com os parâmetros reais do seu projeto e compare com a fatura real dos últimos 2 meses
- ☐ Escreva, em uma frase, o estágio da curva de escalada em que seu projeto está hoje (Mês 1, 2 ou 3 da seção Ilustra)
- ☐ Rode `ponto_de_equilibrio_saas_vs_api` com o preço de uma assinatura SaaS equivalente ao seu caso de uso e descubra se pagar por token ainda é a opção mais barata no seu volume atual

Passo 2 — Como Funciona a Economia de Tokens: Caveman e Headroom

- ☐ Rode `medir_compressao.py` com um parágrafo real que você escreveu para o agente esta semana
- ☐ Aplique `aplicar_caveman()` no mesmo parágrafo e compare o índice de compressão
- ☐ Rode `headroom.sh` sobre a saída de um build ou teste do seu projeto atual
- ☐ Rode `projetar_economia_mensal.py` com os números reais de execuções por dia da sua equipe
- ☐ Preencha a tabela de decisão do capítulo com pelo menos 2 situações reais do seu fluxo de trabalho
- ☐ Identifique 1 lugar onde você aplicaria headroom mas NÃO deveria (ex.: evidência de auditoria)
- ☐ Reescreva em modo caveman o próximo comentário de revisão de código que você receber, antes de repassá-lo ao agente

Passo 3 — Skills Agênticas de Compressão: Caveman, Headroom, Lean-CTX

- ☐ Escolha um log de build recente do seu projeto com mais de 30 linhas e aplique manualmente a regra 3+4 do headroom; confira se a causa raiz continua visível
- ☐ Rode `grep -n` para localizar uma função específica em um arquivo com mais de 200 linhas do seu projeto e leia só a fatia correspondente
- ☐ Escreva no seu `.claude/settings.json` (ou equivalente) uma exceção de path que nunca deve passar por compressão automática
- ☐ Calcule, usando o script do Pilar 1, a economia percentual real de um dos seus próprios logs
- ☐ Rode `grep -n` para um símbolo do seu projeto que você suspeita ter mais de uma ocorrência; se houver ambiguidade, refine a busca por diretório ou contexto antes de decidir qual trecho ler
- ☐ Aplique o auditor do Pilar 4 (ou uma versão simplificada dele) a um log real da sua última sessão de agente e identifique se alguma leitura de arquivo grande pulou o grep

Passo 4 — Cache Inteligente e Memory: RTK-Memory + LiteLLM

- ☐ Rode o `medir_cache_hit.py` com um log real de prompts do seu projeto (substitua o exemplo pelas suas últimas 10-20 chamadas)
- ☐ Identifique, no seu `CLAUDE.md` / `AGENTS.md` atual, qualquer trecho que parece “aprendizado de sessão” e mova para um `RTK-SCRATCHPAD.md`
- ☐ Suba o `litellm-config.yaml` localmente e confirme que o cache semântico responde em poucos milissegundos numa segunda chamada parecida
- ☐ Documente, no seu repositório, a regra “o que muda no prompt-base e o que vai para o scratchpad”
- ☐ Rode o `estimar_economia_cache.py` com os números reais do seu fluxo (volume diário de chamadas, tamanho médio do prefixo, custo por 1k tokens do seu provedor) e registre o valor projetado como meta de acompanhamento mensal
- ☐ Se você opera mais de um agente com prompt próprio, liste onde há trechos de governança duplicados entre eles e planeje extraí-los para um módulo único e referenciado

Passo 5 — Empacotamento de Contexto: Repomix

- ☐ Instale o Repomix com `npx repomix --style xml --output-show-line-numbers` no seu projeto atual
- ☐ Escreva um `.repomix.json` com pelo menos 3 regras de `customPatterns` específicas do seu projeto
- ☐ Rode `medir_ganho_repomix.py` e registre a redução percentual real obtida
- ☐ Versione o `.repomix.json` no repositório e documente, em uma linha, quando ele deve ser revisado
- ☐ Configure o gate de CI de orçamento de tokens (seção Técnica) com um limite calibrado pela calculadora de custo do Capítulo 1, não por um número redondo escolhido de improviso

Passo 6 — Refatoração Estrutural com AST: ast-grep

- ☐ Instale o ast-grep no seu ambiente (`npm install -g @ast-grep/cli` ou o binário nativo da plataforma)
- ☐ Rode `sg run -p '<um padrão do seu projeto>'` e compare o resultado com o mesmo `grep` sobre o mesmo padrão
- ☐ Escreva uma regra YAML de reescrita para uma mudança de assinatura real do seu código e rode com `--update-all` num branch isolado
- ☐ Adicione um step de `sg scan --error` num workflow de CI, mesmo que só em modo de aviso no início

Passo 7 — Otimização de Prompts: DSPy

- ☐ Escolha um prompt do seu projeto que roda pelo menos algumas centenas de vezes por mês
- ☐ Escreva a Signature dele: quais campos realmente entram e quais devem sair
- ☐ Separe de 5 a 10 casos reais já revisados manualmente como exemplos de treino
- ☐ Escreva uma função de métrica objetiva (não “parece certo”, mas um critério comparável)
- ☐ Rode o compilador e compare o tamanho do prompt final com o prompt manual atual
- ☐ Antes de colocar em produção, inspecione o prompt compilado (`dspy.inspect_history`) e confirme que nenhum exemplo de treino carrega dado sensível

Passo 8 — Governança em Produção: 4 Pilares + Checklist

- ☐ Implemente o `middleware_custo.py` sobre uma chamada real do seu projeto e rode por um dia inteiro
- ☐ Rode o `resumo_diario.py` sobre o log gerado e confira se o total agregado bate com o que você esperava gastar naquele dia

- ☐ Ajuste o `limite_usd` do `circuit_breaker_custo.py` para o teto que sua equipe consideraria aceitável por chamada
- ☐ Preencha o `checklist-governanca.yaml` com os nomes reais de aprovador, limite de gasto e canal de alerta da sua equipe
- ☐ Defina, por escrito, qual métrica de qualidade mínima seu circuit breaker vai checar antes de aceitar uma rota de fallback como segura
- ☐ Defina a cadência de recálculo do baseline de alerta (ex.: média móvel de 14 dias) para que o webhook não vire ruído ignorado